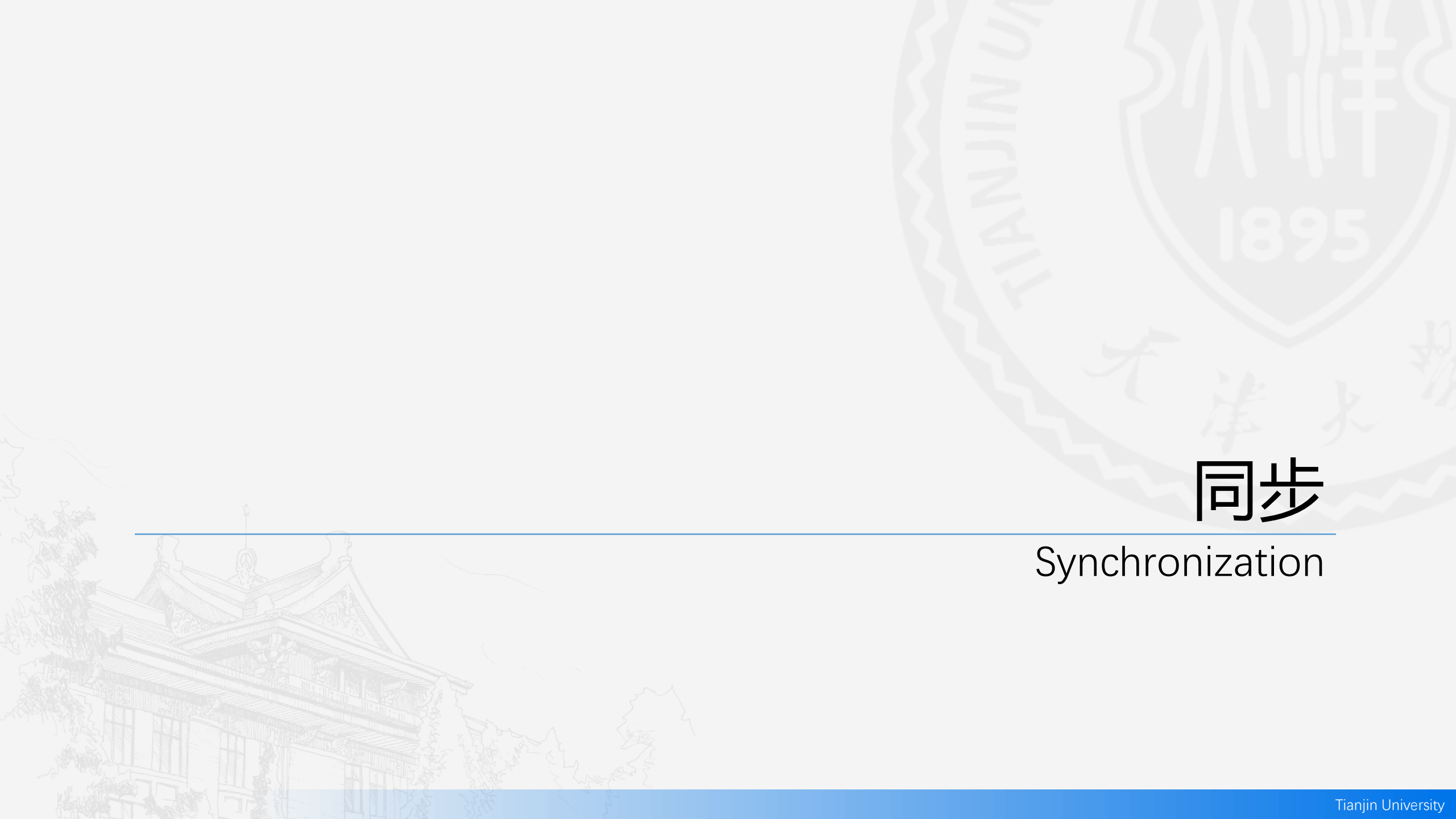




同步

Synchronization



主要内容

- 背景知识 Background
- 竞争条件、临界区问题 The Critical-Section Problem
- 解决方案应满足的条件：互斥、进步、有限等待
- 解决方案：Peterson解决方案 Peterson's Solution、硬件同步 Synchronization Hardware、互斥锁 Mutex Locks、信号量 Semaphores、管程 Monitors
- 经典IPC问题：生产者-消费者问题、读者-写者问题、哲学家就餐问题等。



背景

- 进程或线程可以 并发(Concurrency) 或 并行(Parallelism) 执行
 - 并发指的是在单处理器系统上线程在微观串行执行，而在宏观并发执行。
 - 并行指的是在装配多个处理器的并行计算机系统上，将多个线程分配或者指定到不同的处理器上同时执行。
 - 可随时中断，部分完成执行。
- 同时访问共享数据可能会导致数据不一致
- 保持数据一致性需要机制来确保协作流程的有序执行
- 本章主要讨论协调进程或线程同步 (Synchronization) 和协调 (Coordination)



竞争条件 (Race condition)

■ **竞争条件 (Race condition)**：多个进程/线程并发访问和操作同一数据并且执行结果与特定访问顺序有关。并发或并行执行影响了共享数据的完整性。例：售票问题。

■ count是全局共享变量，含义为剩余票数。

■ 假如现在只有最后一张票，count当前值为1

■ 有2个线程并发执行此段代码

■ 线程A判断 $\text{count} > 0$ 后，在箭头处暂停（可能是时间片用完）

■ 随后线程B运行，也判断出 $\text{count} > 0$ ，随即 $\text{count}--$ ，卖出了一张票，count 变为 0

■ 线程A恢复运行，因为已经判断过 $\text{count} > 0$ 了，所以继续执行 $\text{count}--$ ，卖出了一张票，count 变为 -1

■ **问题核心**：(1)对共享变量先读；(2)再根据读的结果写。(1)(2)两步之前可能被中断。

```
int count;  
  
if ( count > 0 ) {  
    count--;  
    ticket_sold();  
}
```



临界区 (Critical Section / Critical Region)

- **临界资源 (Critical Resource)**：一次仅允许一个进程/线程使用的共享资源。
 - 属于临界资源的硬件有打印机、磁带机等
 - 软件有公共变量、消息队列、共享数组、缓冲区等
- **临界区 (Critical Section)**（有的教材使用 Critical Region，中文翻译相同）
 - 访问临界资源的代码
- 当一个进程/线程在临界区内执行时，其他进程/线程不允许在它们的临界区（访问同一个临界资源）内执行。



临界区问题 (Critical Section Problem)

- 假设某个系统有 n 个进程，每个进程有一段（访问同一临界资源的）**临界区 (Critical Section)**。
- 在进入临界区前，每个进程/线程都必须请求许可。实现这一请求的代码区段称为 **进入区 (entry section)**
- 临界区之后中有 **退出区 (exit section)**
- 其他代码为 **剩余区 (remainder section)**

```
do {  
    进入区 (entry section)  
    临界区 (Critical Section)  
    退出区 (exit section)  
    剩余区 (remainder section)  
} while (true);
```



临界区问题的解决

- 临界区问题的解决方案应满足：
 - **互斥 (mutual exclusion)**：如果某进程在其临界区内执行，那么其他进程都不能在其临界区内执行。
 - **进步 (progress)**：如果没有进程在其临界区内执行，并且有进程需要进入临界区，那么只有那些不在剩余区内执行的进程可以参加选择，以便确定谁能下次进入临界区，而且这种选择不能无限推迟。即：在剩余区的进程不可以阻止其他进程进入临界区。
 - **有限等待 (bounded waiting)**：从一个进程做出进入临界区的请求直到这个请求被允许为止，其他进程允许进入其临界区的次数具有上限。即：任何进程不能永远地被阻止在临界区外。
- 可以假定每个进程的执行速度不为0，但是对于n个进程的相对速度不作任何假设。



内核代码出现竞争条件

- 在任一给定时间点，一个操作系统可能具有多个处于内核态的活动进程。操作系统的内核代码可能出现**竞争条件**。例如：同时维护一个内核数据结构，同时打开一个文件等。
- 处理操作系统的临界区竞争条件问题：**抢占式内核**与**非抢占式内核**
- **非抢占式内核 (nonpreemptive kernel)** 不允许处于内核模式的进程被抢占。
 - 处于内核模式运行的进程会一直运行，直到退出内核模式、阻塞或自愿放弃CPU控制。
 - 基本不会导致竞争条件。
- **抢占式内核 (preemptive kernel)** 允许处于内核模式的进程被抢占。（后面主要讨论抢占式）
 - 响应更快，更适用于实时编程。
 - 需要认真设计，以便确保内核数据结构不会导致竞争条件，尤其是在SMP体系结构中。



临界区问题的解决方案

- Peterson解决方案 (Peterson's Solution)
- 硬件同步 (Synchronization Hardware)
- 互斥锁 (Mutex Locks)
- 信号量 (Semaphores)



Peterson解决方案 (Peterson's Solution)

- 仅用于两个进程/线程交错执行临界区。P0和P1分别循环执行run(i)
- turn和interested[]是共享变量
- 互斥成立：turn不可能同时既等于0又等于1
- 进步要求满足：离开临界区后会将interested[自己]设置为false
- 有限等待要求满足：对方进入临界区后最多一次自己就能进入

```
int turn;
boolean interested[2];

run(int i) {    // i : 0 or 1
    int other = 1 - i;
    interested[i] = true;
    turn = other;
    while (turn == other && interested[other]);
    critical section
    interested[i] = false;
    remainder section
}
```

注意这有一个分号

```
do {
    run(0);
} while(true);
```

P0

```
do {
    run(1);
} while(true);
```

P1



硬件同步 (Synchronization Hardware)

- 特殊硬件指令，用检测和修改字的内容 `test_and_set()`，或者用于原子地 (atomically，即不可被中断地) 交换两个字 `compare_and_swap()`。注意：指令是不可中断的。

```
boolean test_and_set(boolean *target) {  
    boolean rv = *target;  
    *target = true;  
    return rv;  
}
```

```
int compare_and_swap(int *value, int expected,  
                     int new_value) {  
    int temp = *value;  
    if (*value == expected)  
        *value = new_value;  
    return temp;  
}
```

```
do {  
    while(test_and_set(&lock)) ;  
    critical section  
    lock = false;  
} while (true);
```

↑
注意这有一个分号

```
do {  
    while(compare_and_swap(&lock, 0, 1) != 0) ;  
    critical section  
    lock = 0;  
} while (true);
```

↑
注意这有一个分号

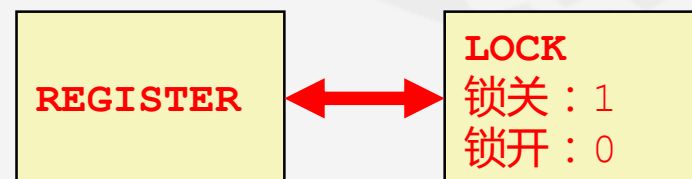


硬件同步 (Synchronization Hardware)

- CPU指令: **TSL指令** 和 **XCHG指令**。注意: 指令是不可中断的。即指令一定可以原子地执行。
- TSL: Test and Set Lock。原子地将内存LOCK (共享) 的值复制到寄存器REGISTER (独占) 中并且同时
将LOCK设置为1 (加锁)。x86上没有TSL指令, 但是有XCHG指令。
- XCHG: Exchange。原子地交换内存LOCK (共享) 和寄存器REGISTER (独占) 中的值。

```
enter_section:
    TSL REGISTER, LOCK
    CMP REGISTER, 0
    JNE enter_section
    critical section
    MOV LOCK, 0
    remainder section
```

```
enter_section:
    MOV REGISTER, 1
    XCHG REGISTER, LOCK
    CMP REGISTER, 0
    JNE enter_section
    critical section
    MOV LOCK, 0
    remainder section
```





互斥锁 (mutex lock)

- 互斥 mutex = mutual exclusion
- 每个互斥锁有一个布尔变量available，函数acquire()获得锁，release()释放锁。对他们的调用必须原子地执行。
- 这种类型的互斥锁也被称为**自旋锁 (spinlock)**，实现方式为**忙等待 (busy waiting)**
- 忙等待的缺点是浪费CPU周期，优点是在等待锁时没有上下文切换的时间。

```
acquire() {  
    while (!available) ;  
    available = false;  
}
```

注意这有一个分号

```
release() {  
    available = true;  
}
```

```
do {  
    acquire();  
    Critical Section  
    release();  
    remainder section  
} while (true);
```



信号量 (Semaphore)

- **信号量 (Semaphore)**：1965年 由荷兰科学家 E. W. Dijkstra 提出。
- **PV操作 (PV operations)**，又叫 **PV原语 (PV atomic operations)**：原子地操作，不可被打断，执行速度非常快。PV是取自荷兰语 Proberen(测试) 和 Verhogen (增加) 的首字母。
- 在各种教材中，P操作又表示为 `down()`、`wait()`，V操作也表示为 `up()`、`signal()`。
- 信号量与PV操作的关系
 - 信号量只能由PV操作使用
 - PV操作的对象只能是信号量
 - 除了定义并被初始化时以外，**信号量不可以在PV操作之外的任何地方被访问**，无论是写还是读，都不可以。



信号量 (Semaphore)

- 信号量分为**记录型信号量**和**非记录型信号量**，非记录型又被称为**计数信号量 (counting semaphore)**或**整型信号量**，**二进制 (二元) 信号量 (binary semaphore)**是计数信号量的特例。

	记录型信号量	计数信号量	二元信号量
信号量是什么	一个整数及一个等待队列	一个非负整数	一个整数 (值仅0和1)
信号量值为正整数	代表相应的资源可用，值为可用的资源数量		
信号量值为0	代表无可用的相应资源，且没有等待使用该类型资源的进程	代表无可用的相应资源，有没有等待使用该类型资源的进程不详	
信号量值为负整数	代表相应的资源不可用，且有等待使用该类型资源的进程，信号量的绝对值为等待进程的个数	不允许取负整数	
相应PV操作	对应“睡眠与唤醒”版本的PV操作	对应“忙等待”版本的PV操作	



PV操作“忙等待 (busy waiting)”版本的实现

- PV操作分为忙等待 (busy waiting) 版本和睡眠与唤醒 (sleep and wakeup) 版本。
- 忙等待 (busy waiting) 版本的PV操作适用于计数信号量 (包括二元信号量)
- 计数信号量中只包括一个整数value。
value的取值范围为非负整数。

```
struct semaphore {  
    int value;  
};
```

```
P(semaphore *sem) {  
    while (!sem->value>0) {  
        yield(); // 主动放弃CPU, 当前进程转入就绪态  
    }  
    sem->value--;  
}
```

```
V(semaphore *sem) {  
    sem->value++;  
}
```



$S = 1$

Process A:

... ..

P(S);

Print(a.doc);

V(S);

... ..

Runnable

```
P (Semaphore s){  
    while (!s>0)  
        yield;  
    s--;  
}
```

```
V (Semaphore s){  
    s++;  
}
```



$S = 1 \rightarrow 0$

Process A:

```
... ..  
P(S);  
Print(a.doc);  
V(S);  
... ..
```

Runnable

```
P (Semaphore s){  
    while (!s>0)  
        yield;  
    s--;  
}
```

```
V (Semaphore s){  
    s++;  
}
```



$S = 1 \rightarrow 0$

Process A:

```
... ..  
P(S);  
Print(a.doc);  
V(S);  
... ..
```

Runnable

Process B:

```
... ..  
P(S);  
Print(b.doc);  
V(S);  
... ..
```

Runnable

```
P (Semaphore s){  
    while (!s>0)  
        yield;  
    s--;  
}
```

```
V (Semaphore s){  
    s++;  
}
```



$S = 1 \rightarrow 0$

Process A:

```
... ..  
P(S);  
Print(a.doc);  
V(S);  
... ..
```

Runnable

Process B:

```
... ..  
P(S);  
Print(b.doc);  
V(S);  
... ..
```

Runnable

Process C:

```
... ..  
P(S);  
Print(c.doc);  
V(S);  
... ..
```

Runnable

```
P (Semaphore s){  
    while (!s>0)  
        yield;  
    s--;  
}
```

```
V (Semaphore s){  
    s++;  
}
```



$S = 1 \rightarrow 0$

Process A:

```
... ..  
P(S);  
Print(a.doc);  
V(S);  
... ..
```

Runnable

Process B:

```
... ..  
P(S);  
Print(b.doc);  
V(S);  
... ..
```

Runnable

Process C:

```
... ..  
P(S);  
Print(c.doc);  
V(S);  
... ..
```

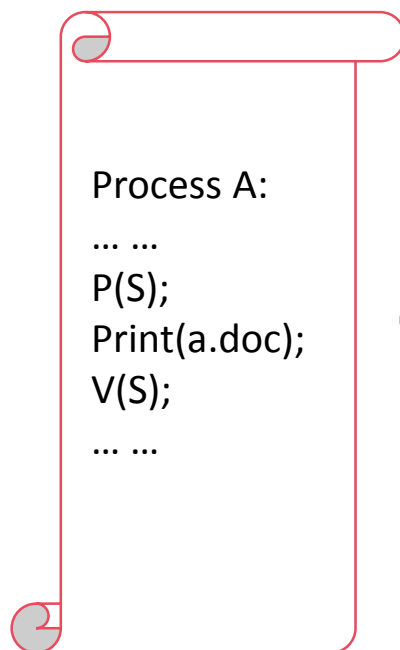
Runnable

```
P (Semaphore s){  
    while (!s>0)  
        yield;  
    s--;  
}
```

```
V (Semaphore s){  
    s++;  
}
```

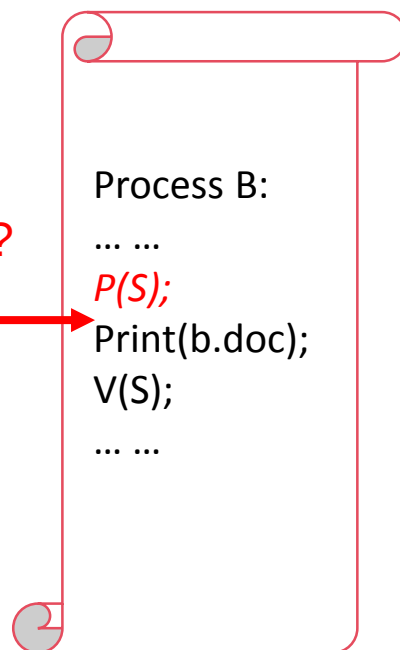


$S = 1 \rightarrow 0 \rightarrow 1$



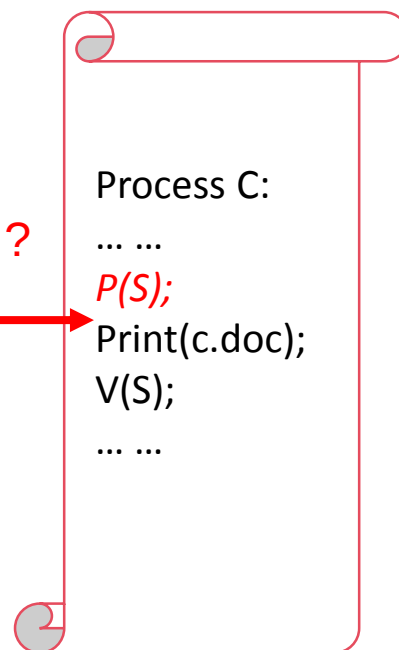
Runnable

?



Runnable

?



Runnable

```
P (Semaphore s){  
    while (!s>0)  
        yield;  
    s--;  
}
```

```
V (Semaphore s){  
    s++;  
}
```



PV操作“睡眠与唤醒 (sleep and wakeup)”版本的实现

- 睡眠与唤醒 (sleep and wakeup) 版本的PV操作适用于记录型信号量。
- 记录型信号量中包括一个整数value和一个进程等待队列wait_queue。value的取值范围为整个整数集。

```
struct semaphore {  
    int value;  
    struct task_struct *wait_queue;  
};
```

```
P(semaphore *sem) {  
    sem->value--;  
    if (sem->value < 0) {  
        block(); // 当前进程转入阻塞态  
        // 并将当前进程加入 sem->wait_queue 队尾  
    }  
}
```

```
V(semaphore *sem) {  
    sem->value++;  
    if (sem->value <= 0) {  
        // 从 sem->wait_queue 队首取出一个进程 p  
        wakeup(p); // 并将进程 p 的状态转为就绪态  
    }  
}
```



$S = 1$

Process A:

... ..

P(S);

Print(a.doc);

V(S);

... ..

Runnable

```
P (Semaphore s){  
    s=s-1;  
    if (s<0){  
        sleep...;  
    }  
}
```

```
V (Semaphore s){  
    s=s+1;  
    if (s<=0){  
        wakeup...;  
    }  
}
```



$S = 1 \rightarrow 0$

Process A:

```
... ..  
P(S);  
Print(a.doc);  
V(S);  
... ..
```

Runnable



```
P (Semaphore s){  
    s=s-1;  
    if (s<0){  
        sleep...;  
    }  
}
```

```
V (Semaphore s){  
    s=s+1;  
    if (s<=0){  
        wakeup...;  
    }  
}
```



$S = 1 \rightarrow 0$

Process A:

```
... ..  
P(S);  
Print(a.doc);  
V(S);  
... ..
```

Runnable

Process B:

```
... ..  
P(S);  
Print(b.doc);  
V(S);  
... ..
```

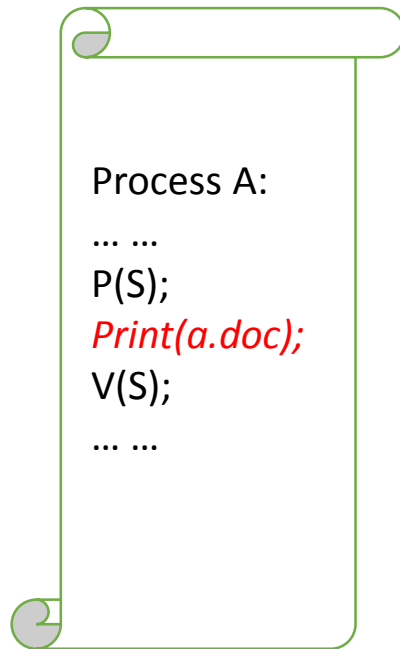
Runnable

```
P (Semaphore s){  
    s=s-1;  
    if (s<0){  
        sleep...;  
    }  
}
```

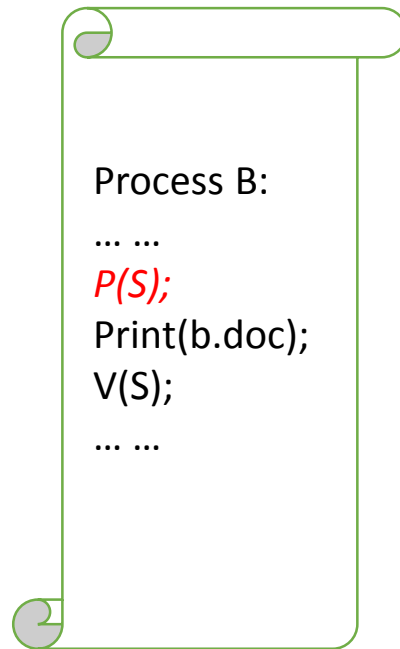
```
V (Semaphore s){  
    s=s+1;  
    if (s<=0){  
        wakeup...;  
    }  
}
```



$S = 1 \rightarrow 0 \rightarrow -1$



Runnable



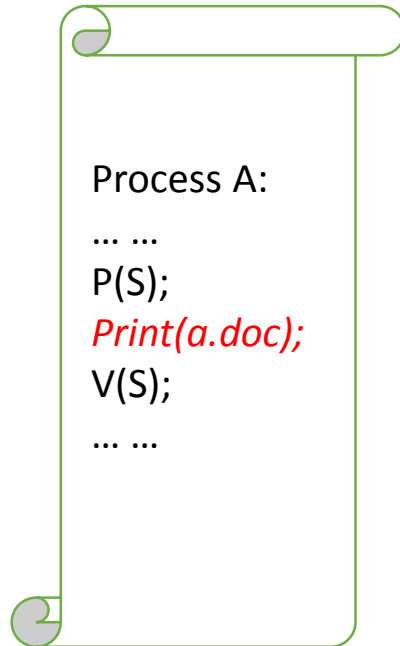
Blocked

```
P (Semaphore s){  
    s=s-1;  
    if (s<0){  
        sleep...;  
    }  
}
```

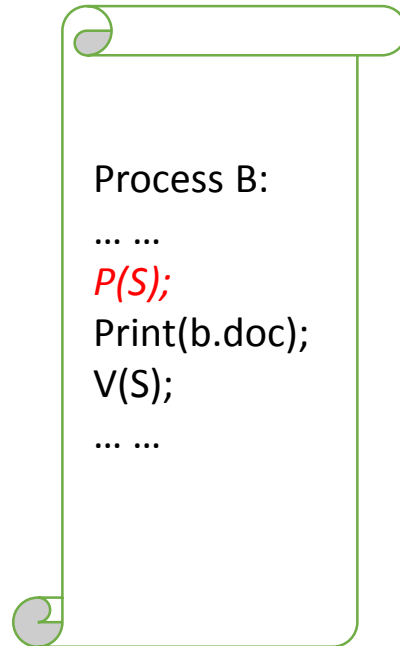
```
V (Semaphore s){  
    s=s+1;  
    if (s<=0){  
        wakeup...;  
    }  
}
```



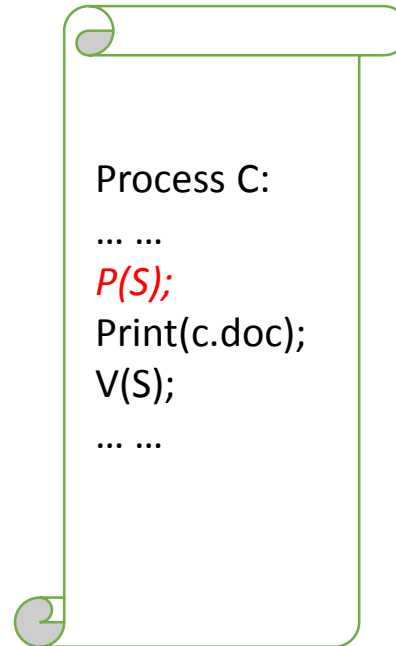
$S = 1 \rightarrow 0 \rightarrow -1$



Runnable



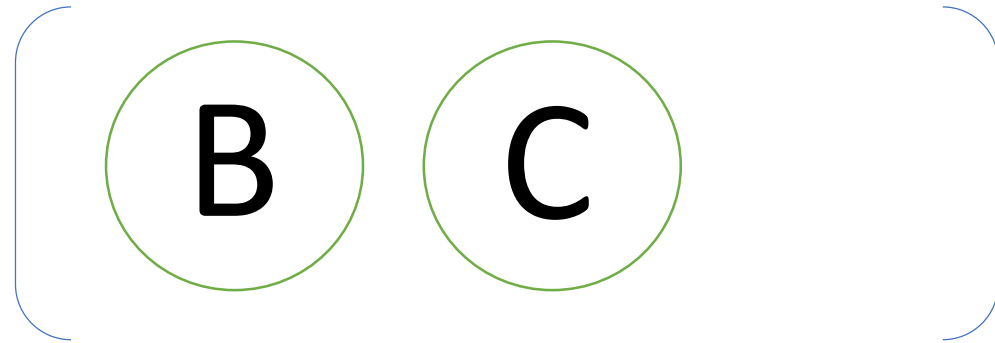
Blocked



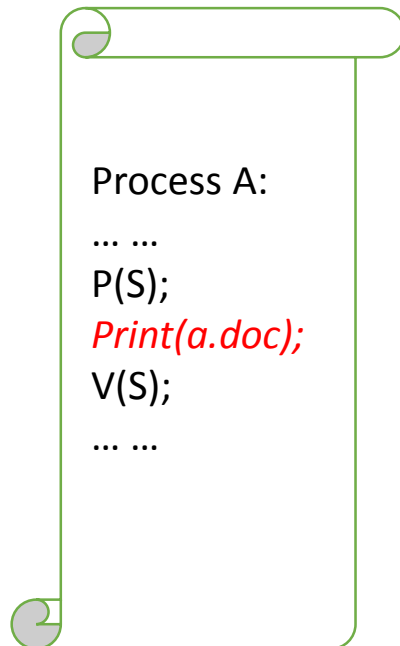
Runnable

```
P (Semaphore s){  
    s=s-1;  
    if (s<0){  
        sleep...;  
    }  
}
```

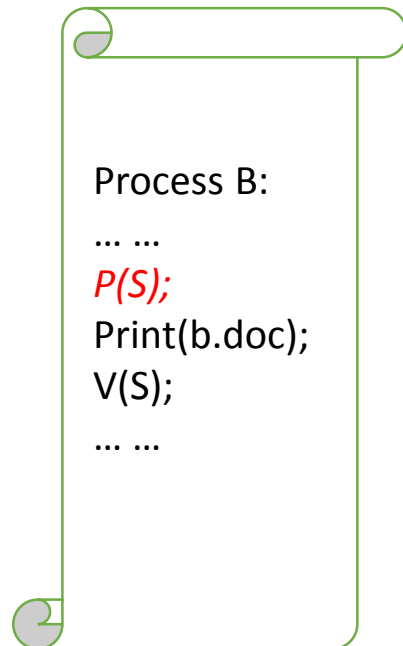
```
V (Semaphore s){  
    s=s+1;  
    if (s<=0){  
        wakeup...;  
    }  
}
```



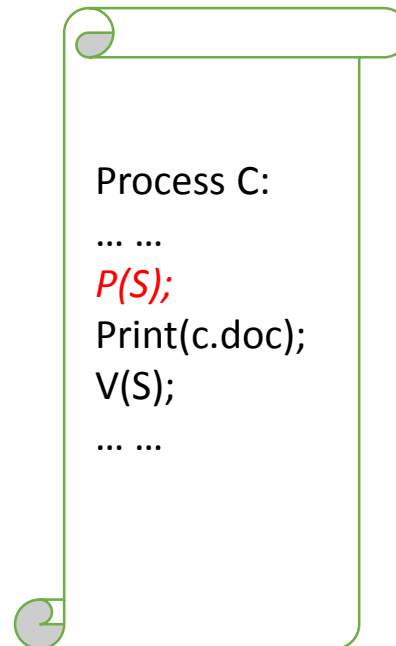
$S = 1 \rightarrow 0 \rightarrow -1 \rightarrow -2$



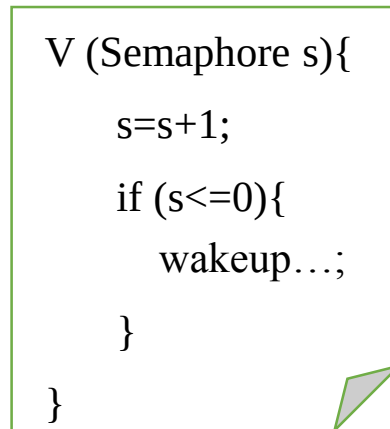
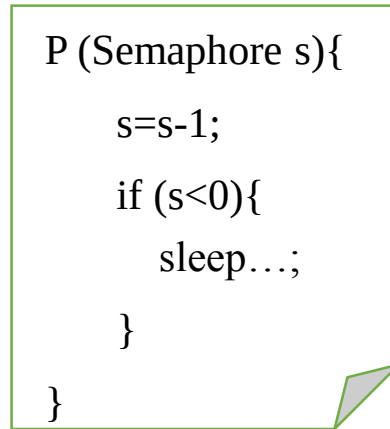
Runnable



Blocked

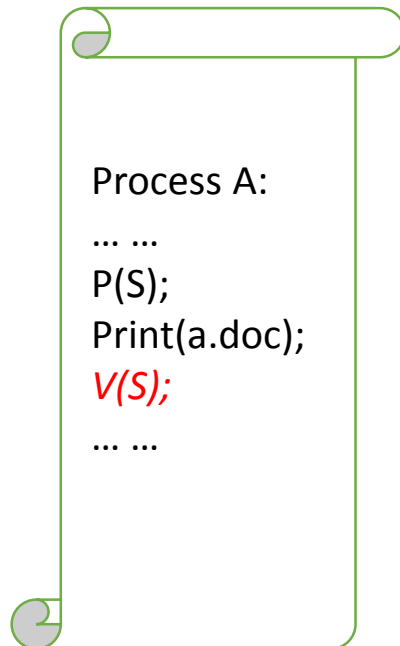
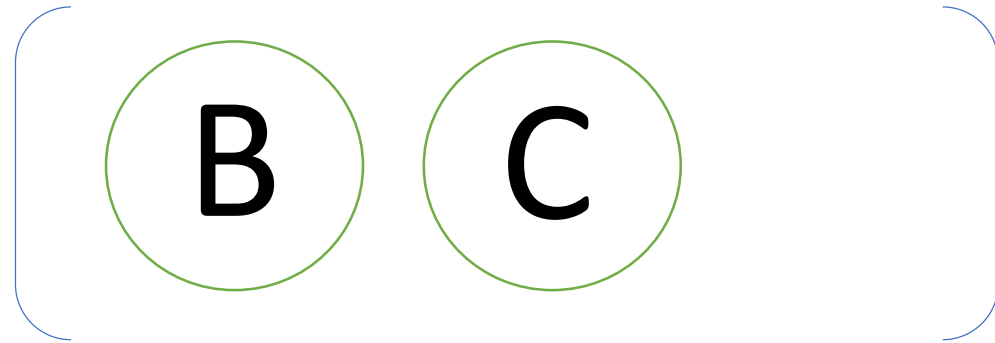


Blocked





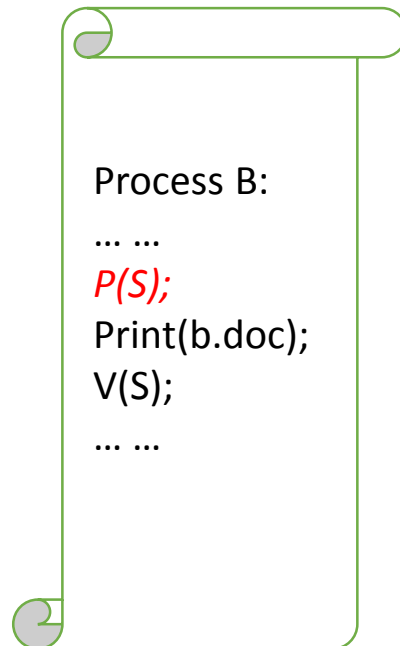
$S = 1 \rightarrow 0 \rightarrow -1 \rightarrow -2$



Process A:

```
... ..  
P(S);  
Print(a.doc);  
V(S);  
... ..
```

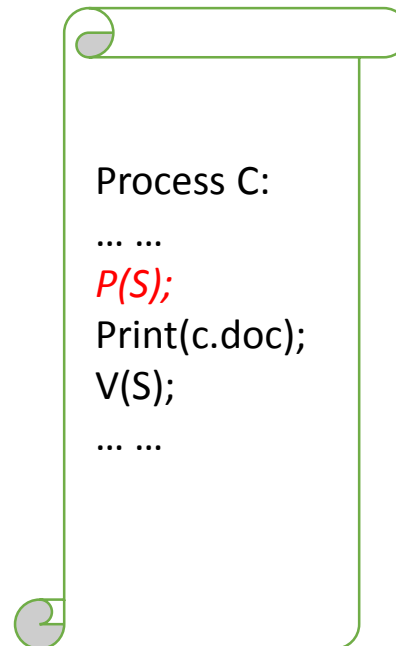
Runnable



Process B:

```
... ..  
P(S);  
Print(b.doc);  
V(S);  
... ..
```

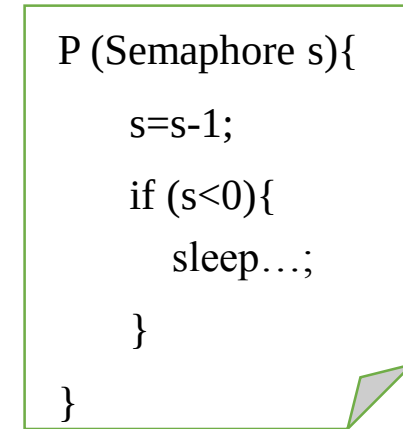
Blocked



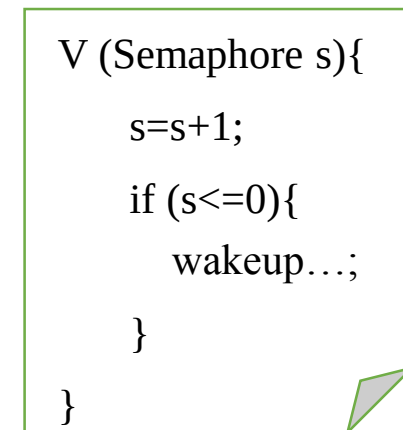
Process C:

```
... ..  
P(S);  
Print(c.doc);  
V(S);  
... ..
```

Blocked



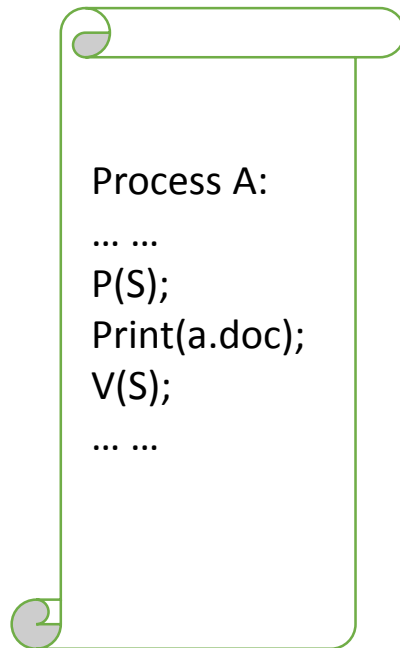
```
P (Semaphore s){  
    s=s-1;  
    if (s<0){  
        sleep...;  
    }  
}
```



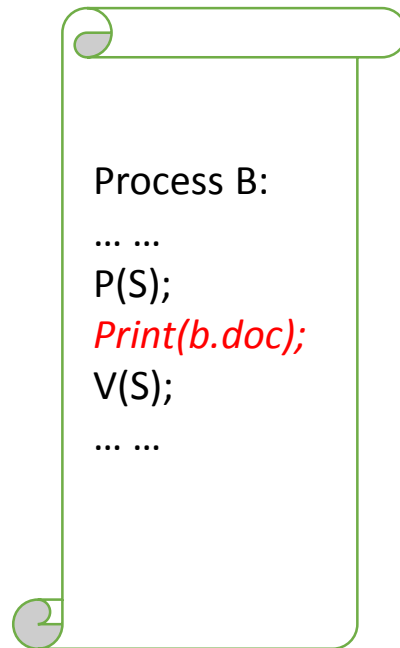
```
V (Semaphore s){  
    s=s+1;  
    if (s<=0){  
        wakeup...;  
    }  
}
```



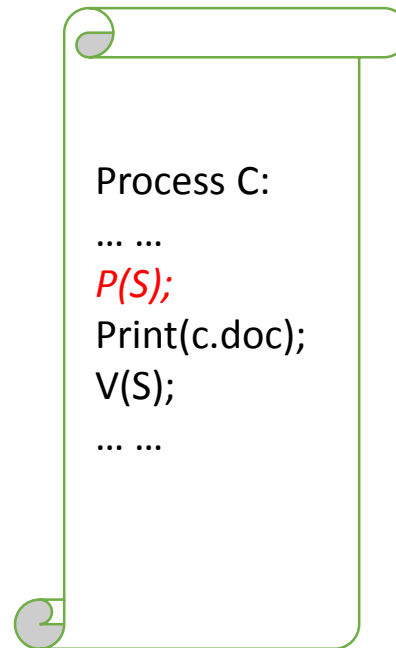
$S = 1 \rightarrow 0 \rightarrow -1 \rightarrow -2 \rightarrow -1$



Runnable



Runnable



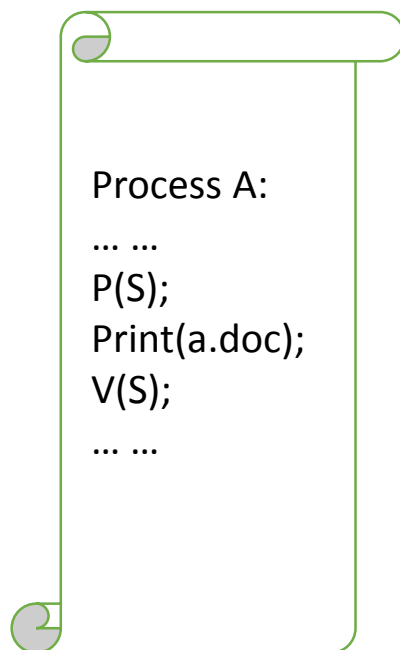
Blocked

```
P (Semaphore s){  
    s=s-1;  
    if (s<0){  
        sleep...;  
    }  
}
```

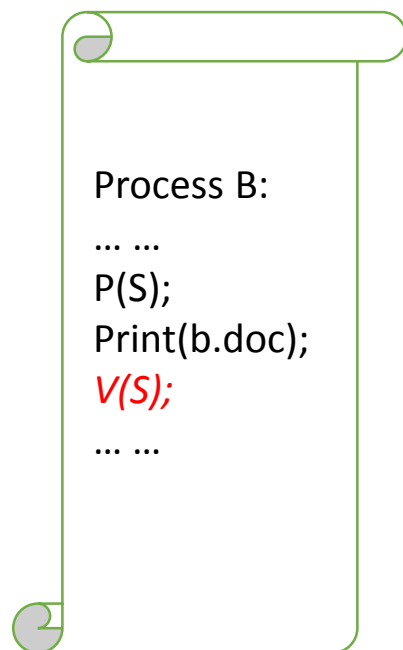
```
V (Semaphore s){  
    s=s+1;  
    if (s<=0){  
        wakeup...;  
    }  
}
```



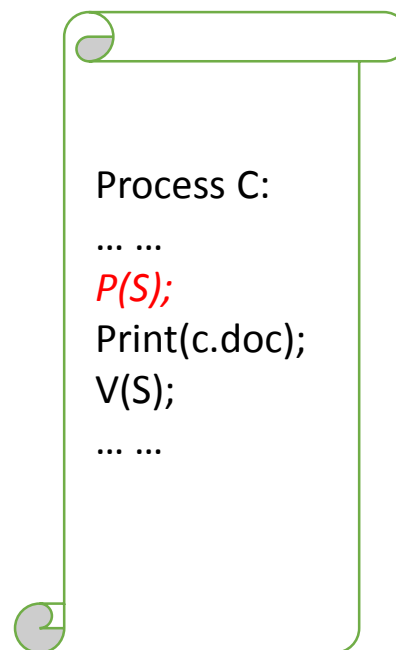
$S = 1 \rightarrow 0 \rightarrow -1 \rightarrow -2 \rightarrow -1$



Runnable



Runnable



Blocked

```
P (Semaphore s){  
    s=s-1;  
    if (s<0){  
        sleep...;  
    }  
}
```

```
V (Semaphore s){  
    s=s+1;  
    if (s<=0){  
        wakeup...;  
    }  
}
```



$S = 1 \rightarrow 0 \rightarrow -1 \rightarrow -2 \rightarrow -1 \rightarrow 0$

Process A:

```
... ..  
P(S);  
Print(a.doc);  
V(S);  
... ..
```

Runnable

Process B:

```
... ..  
P(S);  
Print(b.doc);  
V(S);  
... ..
```

Runnable

Process C:

```
... ..  
P(S);  
Print(c.doc);  
V(S);  
... ..
```

Runnable

```
P (Semaphore s){  
    s=s-1;  
    if (s<0){  
        sleep...;  
    }  
}
```

```
V (Semaphore s){  
    s=s+1;  
    if (s<=0){  
        wakeup...;  
    }  
}
```



$S = 1 \rightarrow 0 \rightarrow -1 \rightarrow -2 \rightarrow -1 \rightarrow 0$

Process A:

```
... ..  
P(S);  
Print(a.doc);  
V(S);  
... ..
```

Runnable

Process B:

```
... ..  
P(S);  
Print(b.doc);  
V(S);  
... ..
```

Runnable

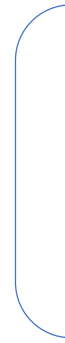
Process C:

```
... ..  
P(S);  
Print(c.doc);  
V(S);  
... ..
```

Runnable

```
P (Semaphore s){  
    s=s-1;  
    if (s<0){  
        sleep...;  
    }  
}
```

```
V (Semaphore s){  
    s=s+1;  
    if (s<=0){  
        wakeup...;  
    }  
}
```



$S = 1 \rightarrow 0 \rightarrow -1 \rightarrow -2 \rightarrow -1 \rightarrow 0 \rightarrow 1$

Process A:

```
... ..  
P(S);  
Print(a.doc);  
V(S);  
... ..
```

Runnable

Process B:

```
... ..  
P(S);  
Print(b.doc);  
V(S);  
... ..
```

Runnable

Process C:

```
... ..  
P(S);  
Print(c.doc);  
V(S);  
... ..
```

Runnable

```
P (Semaphore s){  
    s=s-1;  
    if (s<0){  
        sleep...;  
    }  
}
```

```
V (Semaphore s){  
    s=s+1;  
    if (s<=0){  
        wakeup...;  
    }  
}
```



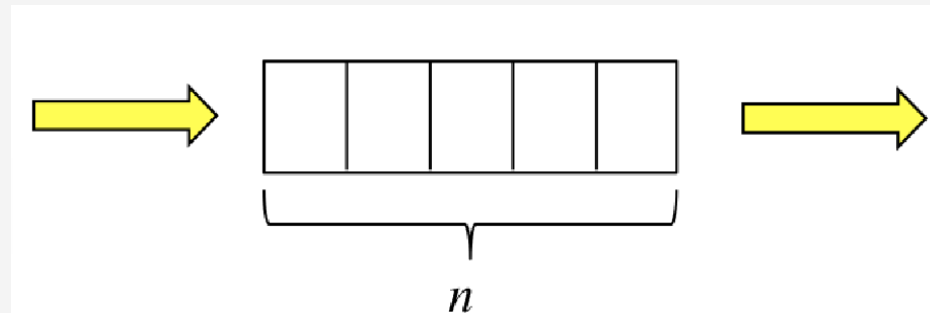
经典IPC问题 (Classical IPC Problem)

- 经典IPC问题 (Classical Inter Process Communication Problem)
- 生产者-消费者问题
- 哲学家就餐问题
- 读者-写者问题 (读者优先、写者优先、公平竞争)



生产者-消费者问题 (Producer-Consumer Problem)

- 生产者消费者问题 (Producer-consumer problem)，也称有限缓冲问题 (Bounded-buffer problem)
- 该问题描述了共享固定大小缓冲区的两个线程——“生产者”和“消费者”运行时的问题。生产者的主要作用是生成数据放到缓冲区中，消费者在缓冲区消耗这些数据。该问题的关键就是要保证生产者不会在缓冲区满时加入数据，消费者也不会缓冲区中空时消耗数据。
- 要解决该问题，就必须让生产者在缓冲区满时休眠，等到下次消费者消耗缓冲区中的数据的时候，生产者才能被唤醒，开始往缓冲区添加数据。同样，也必须让消费者在缓冲区空时进入休眠，等到生产者往缓冲区添加数据之后，再唤醒消费者。





生产者-消费者问题 (Producer-Consumer Problem)

■ 这是一个存在严重竞争条件的错误解法。



```
int N; // 缓冲区大小
int count = 0; // 缓冲区内物品数量
```

```
void producer() { // 生产者
    int item;
    while (true) {
        item = produce_item();
        if (count==N) sleep();
        insert_item(item);
        count = count + 1;
        if (count==1) wakeup(consumer);
    }
}
```

```
void consumer() { // 消费者
    int item;
    while (true) {
        if (count==0) sleep();
        item = remove_item();
        count = count - 1;
        if (count==N-1) wakeup(producer);
        consume_item(item);
    }
}
```

试想以下场景：(1)消费者先到，由于判断到count为0，将转入阻塞态，在即将但还未执行sleep()时，时间片用完，暂时放弃CPU转入就绪态；(2)生产者到，顺利向缓冲区中放入一个面包，最后count为1，尝试唤醒消费者，但是消费者并没有睡眠，它在就绪态！唤醒无效。(3)消费者得到时间片，开始运行，执行sleep()，转入阻塞态。(4)生产者顺利的生产了N个面包，最终把自己阻塞。(5)生产者和消费者双双死锁！



生产者-消费者问题 (Producer-Consumer Problem)

■ 这是一个正确使用PV操作的解法。

```
semaphore mutex = 1; // 缓冲区信号量  
semaphore empty = N; // 空格子信号量  
semaphore full = 0;  // 满格子信号量
```

```
void producer() { // 生产者  
    int item;  
    while (true) {  
        item =  
produce_item();  
        P(empty);  
        P(mutex);  
        insert_item(item);  
        V(mutex);  
        V(full);  
    }  
}
```

```
void consumer() { // 消费者  
    int item;  
    while (true) {  
        P(full);  
        P(mutex);  
        item = remove_item();  
        V(mutex);  
        V(empty);  
        consume_item(item);  
    }  
}
```





生产者-消费者问题 (Producer-Consumer Problem)

- 这是一个**错误使用PV操作**的解法。
- **错误的原因是不正确的P操作顺序**
- 考虑下面的场景：(1)消费者先到，拿到了缓冲区，拿不到满格子；
- (2)生产者后到，拿到了空格子，拿不到缓冲区；
- (3) **生产者和消费者双双死锁！**
- **V操作顺序不会引起死锁**



```
semaphore mutex = 1; // 缓冲区信号量
semaphore empty = N; // 空格子信号量
semaphore full = 0; // 满格子信号量
```

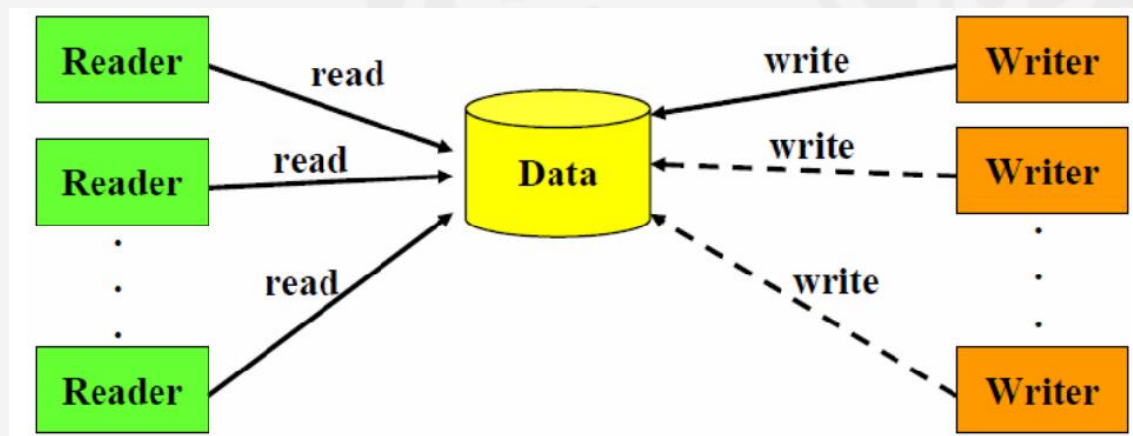
```
void producer() { // 生产者
    int item;
    while (true) {
        item =
produce_item();
        P(empty);
        P(mutex);
        insert_item(item);
        V(mutex);
        V(full);
    }
}
```

```
void consumer() { // 消费者
    int item;
    while (true) {
        P(mutex);
        P(full);
        item = remove_item();
        V(mutex);
        V(empty);
        consume_item(item);
    }
}
```



读者-写者问题 (Readers and Writers Problem)

- 读者写者问题可以这样的描述，有一群写者和一群读者，写者在写同一本书，读者也在读这本书。多个读者可以同时读这本书，但是，只能有一个写者在写书，而且写者在写书时读者也不可以读书。
- 根据实现方案的不同，读者-写者问题又分为“读者优先”、“写者优先”、“公平竞争”三种情况。





读者-写者问题（读者优先）

```
semaphore fmutex=1, rdcntmutex=1;  
int reader_count = 0;
```

到达: R1, R2, W1, R3, R4, W2, R5 依次到达齐
离开: (R1, R2, R3, R4, R5), W1, W2
(离开序列中的括号内离开顺序随意)

```
void reader(){  
    while ( TRUE ){  
        P(rdcntmutex);  
        if( reader_count == 0 ) { P(fmutex); }  
        reader_count = reader_count + 1;  
        V(rdcntmutex);  
        // Do read operation ...  
        P(rdcntmutex);  
        reader_count = reader_count - 1;  
        if( reader_count == 0 ) { V(fmutex); }  
        V(rdcntmutex);  
    }  
}
```

```
void writer(){  
    while ( TRUE ){  
        P(fmutex);  
        // Do write operation ...  
        V(fmutex);  
    }  
}
```



读者-写者问题（写者优先）

```
semaphore fmutex=1, rdcntmutex=1, wtcntmutex=1, queue=1;  
int reader_count = 0, writer_count = 0;
```

到达: R1, R2, W1, R3, R4, W2, R5 依次到达齐
离开: (R1, R2), W1, W2, (R3, R4, R5)

```
void reader(){  
    while( TRUE ){  
        P(queue);  
        P(rcntmutex);  
        if( reader_count == 0 ) { P(fmutex); }  
        reader_count = reader_count + 1;  
        V(rcntmutex);  
        V(queue);  
        //Do read operation ...  
        P(rcntmutex);  
        reader_count = reader_count - 1;  
        if( reader_count == 0 ) { V(fmutex); }  
        V(rcntmutex);  
    }  
}
```

```
void writer(){  
    while( TRUE ){  
        P(wtcntmutex);  
        if( writer_count == 0 ) { P(queue); }  
        writer_count = writer_count + 1;  
        V(wtcntmutex);  
        P(fmutex);  
        //Do write operation ...  
        V(fmutex);  
        P(wtcntmutex);  
        writer_count = writer_count - 1;  
        if( writer_count == 0 ) { V(queue); }  
        V(wtcntmutex);  
    }  
}
```



读者-写者问题（公平竞争）

```
semaphore fmutex=1, rdcntmutex=1, queue=1;  
int reader_count = 0;
```

到达: R1, R2, W1, R3, R4, W2, R5 依次到达齐
离开: (R1, R2), W1, (R3, R4), W2, R5

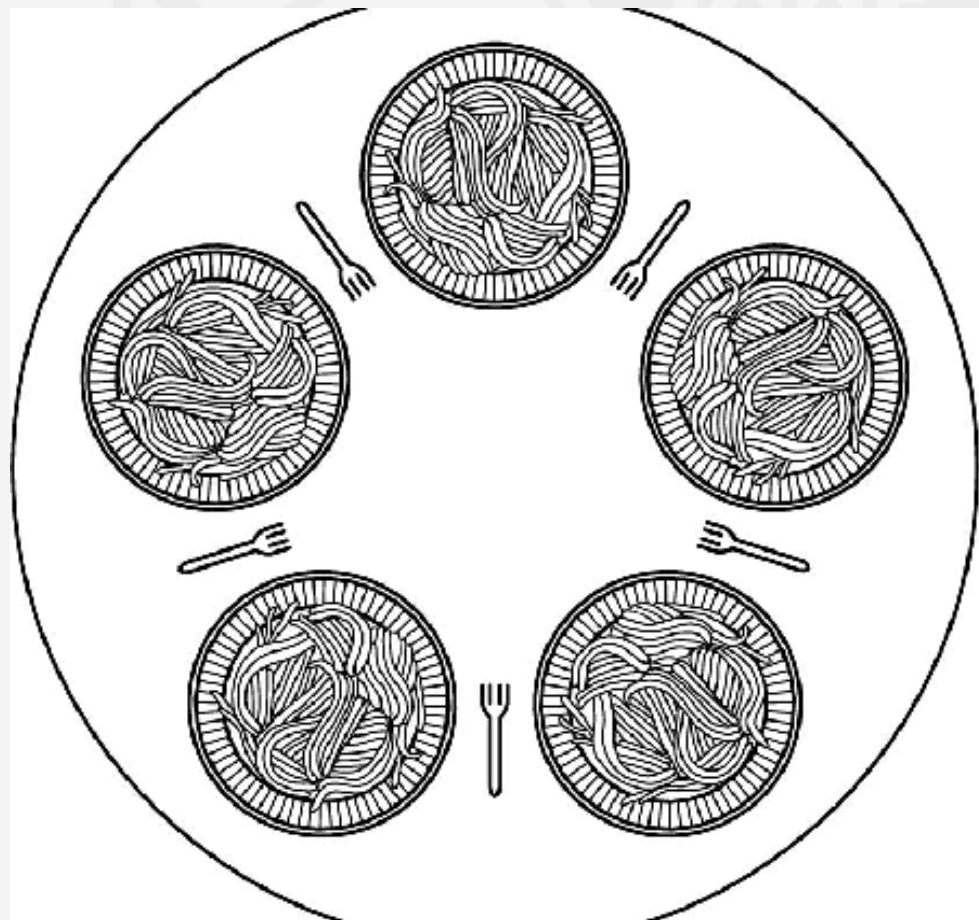
```
void reader(){  
    while( TRUE ){  
        P(queue);  
        P(rcntmutex);  
        if( reader_count == 0 ) { P(fmutex); }  
        reader_count = reader_count + 1;  
        V(rcntmutex);  
        V(queue);  
        //Do read operation ...  
        P(rcntmutex);  
        reader_count = reader_count - 1;  
        if( reader_count == 0 ) { V(fmutex); }  
        V(rcntmutex);  
    }  
}
```

```
void writer(){  
    while( TRUE ){  
        P(queue);  
        P(fmutex);  
        V(queue);  
        //Do write operation ...  
        V(fmutex);  
    }  
}
```



哲学家就餐问题 (Dining Philosophers Problem)

- 哲学家就餐问题可以这样表述，假设有五位哲学家围坐在一张圆形餐桌旁，做以下两件事情之一：吃饭，或者思考。吃东西的时候，他们就停止思考，思考的时候也停止吃东西。
- 餐桌中间有一大碗意大利面，每两个哲学家之间有一只餐叉。因为用一只餐叉很难吃到意大利面，所以假设哲学家必须用两只餐叉吃东西。他们只能使用自己左右手边的那两只餐叉。哲学家就餐问题有时也用米饭和筷子而不是意大利面和餐叉来描述，因为很明显，吃米饭必须用两根筷子。





哲学家就餐问题 (Dining Philosophers Problem)

- 一种避免竞争的方法是一人吃饭锁住整个桌子，这确实可以避免竞争，但是效率太低，因为其实非相邻的二位哲学家是可以同时吃饭的。
- 另一个**错误**的解决方案是，所有想吃饭的哲学家，都要先尝试拿起左侧的筷子，如果成功，再尝试拿起右侧的筷子，如果又成功了，就可以吃饭了。这个过程中如果任何一步失败了，就反复重试直到成功为止。极端的情况是所有人拿着自己左边的筷子，尝试去拿自己右边的筷子失败，**死锁**。
- 上述方案的如下改进方案仍然是**错误**的，如果拿起左边筷子成功而拿起右边筷子失败，则先放下左边筷子，过一会再从头尝试。极端的情况是所有人拿起自己左边的筷子成功，尝试去拿自己右边的筷子失败，于是所有人同时放下左边的筷子，经过**相同的时间间隔**，重复上述过程，反复失败，**活锁**。

```
int N = 5;

void philosopher(int i) {
    while(true) {
        take_fork(i);
        take_fork((i+1)%N);
        eat(i);
        put_fork(i);
        put_fork((i+1)%N);
    }
}
```





哲学家就餐问题 (Dining Philosophers Problem)

```
#define N 5
#define LEFT (i+N-1)%N
#define RIGHT (i+1)%N
#define THINKING 0
#define HUNGRY 1
#define EATING 2

int state[N];
semaphore mutex = 1;
semaphore s[N];
```

```
void philosopher(int i)
{
    while(true) {
        think(i);
        take_fork(i);
        eat(i);
        put_fork(i);
    }
}
```

```
void take_fork(int i)
{
    P(mutex);
    state[i]=HUNGRY;
    test(i);
    V(mutex);
    P(s[i]);
}
```

```
void put_fork(int i)
{
    P(mutex);

    state[i]=THINKING;
    test(LEFT);
    test(RIGHT);
    V(mutex);
}
```

```
void test(int i) {
    if(state[i]==HUNGRY && state[LEFT]!=EATING && state[RIGHT]!=EATING) {
        state[i]=EATING;
        V(s[i]);
    }
}
```





管程 (Monitor)

- 虽然信号量提供了一种方便且有效的进程同步机制，但是它们的错误使用可能导致难以检测的时序错误。例如，生产者-消费者问题中对调了相邻的P操作。
- **管程 (Monitor)** 是一个由过程、变量及数据结构等组成的集合。进程可以调用管程中的过程，但是管程之外声明的过程，不可以直接访问管程内的数据结构。
- 管程的一个重要特性是**任一时刻管程中只能有一个活跃进程**，这一特性使管程能有效完成互斥。
- **管程是编译语言的组成部分，进入管程时的互斥由编译器负责**。通常的做法是用一个互斥量或二元信号量。因为是编译器而非程序员安排互斥，程序员无需关心编译器是如何实现互斥的，只需要将所有的临界区转换成管程即可，决不会有两个进程同时执行临界区的代码。
- **管程需要语言编译器支持**，如 `Java`，使用 `synchronized` 关键词保证互斥。



管程 (Monitor)

- 除了互斥关系，管程进程还需要协调同步。管程中的解决方案是引入**条件变量 (condition variables)** 以及两个操作 **wait()** 和 **signal()**。
- 当一个管程过程发现它无法继续运行时，例如生产者-消费者问题中，生产者发现缓冲区满了，它就会在某个条件变量（如full）上执行wait()操作。wait()操作导致进程自身阻塞，并将另一个管程进程调入运行。
- 另一个进程，如消费者，可以通过对一个条件变量执行signal()操作，唤醒因等待此条件变量而正在睡眠的伙伴进程，即生产者。
- 管程的wait/signal操作和信号量的wait/signal操作是不同的。条件变量不是计数器，不能像信号量那样积累信号以便以后使用。所以，如果向一个条件变量发送信息，但是这个条件变量上并没有等待进程，则这个信号会永远丢失。即wait()操作必须在signal()操作之前。



管程 (Monitor)

```
monitor ProducerConsumer
    condition full, empty; //条件变量用来实现同步
    int count=0; // 缓冲区中的产品数
    void insert(Item item){ //把产品item放入缓冲区
        if(count == N) wait(full);
        count++;
        insert_item(item);
        if(count == 1) signal(empty);
    }
    Item remove(){ //从缓冲区取出一个产品
        if(count==0) wait(empty);
        count--;
        Item item = remove_item();
        if(count == N-1) signal(full);
        return item;
    }
end monitor;
```

- 觉得与“存在严重竞争的sleep/wakeup方案”很像?
- 区别是: **管程自动保证insert和remove不会被中断**

```
producer() {
    while(true) {
        item = produce_item();
        ProducerConsumer.insert(item);
    }
}
consumer() {
    while(true) {
        item = ProducerConsumer.remove();
        consume_item(item);
    }
}
```



课后阅读

- 《操作系统概念 OSC（第9版）》第6章，6.1 – 6.8节。
- 《现代操作系统 MOS（第4版）》第2章，2.3节、2.5节。
- 《深入理解计算机系统 CSAPP（第3版）》第12章。